

Cours complet : CLI, n8n et n8n CLI

Document de formation — version 1.2.40 de `n8ncli` Tous les exemples proviennent du dépôt `C:\Users\ema\n8n-projects`.

Sommaire

Partie 1 — La CLI (fondamentaux)

1. Qu'est-ce qu'une CLI
2. Le terminal, le shell, le programme
3. Anatomie d'une commande
4. STDIN / STDOUT / STDERR et codes de sortie
5. Pipes et redirections
6. PATH, variables d'environnement

Partie 2 — n8n (le serveur)

7. Qu'est-ce que n8n
8. Les concepts fondamentaux
9. Le modèle de données
10. Les credentials
11. Le cycle de vie d'un workflow
12. Où sont stockées les données

Partie 3 — n8n CLI (le client de développement)

13. Le problème que ça résout
14. Workflow-as-Code
15. Architecture : les 4 couches
16. Le cycle de vie du développement
17. Le catalogue de commandes
18. Codes de sortie et automatisation
19. Les standards imposés

Partie 4 — Sécurité

20. Le modèle de menace
21. Où sont les secrets
22. Les 6 vrais points de risque
23. La checklist de sécurité

Partie 5 — Pratique

24. Exercices guidés

25. Le mémo

PARTIE 1 — LA CLI

Module 1 — Qu'est-ce qu'une CLI

Définition

CLI = Command Line Interface (interface en ligne de commande). Un programme avec lequel on **tape du texte** au lieu de cliquer.

L'analogie du restaurant

	Restaurant (GUI)	Distributeur (CLI)
Commander	Lire le menu, pointer sur l'écran	Taper le code sur le clavier
Choix possibles	Ce que le menu propose	Ce que le clavier permet (infini)
Rapidité	Lents, 3 clics par action	1 ligne = 1 action complète
Mémorisation	Il faut tout voir	5 à 10 commandes suffisent
Automatisation	Impossible	Possible

La définition qui compte

Une CLI n'est pas juste « un programme en texte ». C'est un programme conçu selon 5 règles :

1. TEXTE EN SORTIE

→ la sortie est lisible et copiable
2. CODE DE SORTIE

→ 0 ou 1, jamais d'ambiguïté
3. PARAMÈTRES

→ pas de menus cachés, tout est explicite
4. LECTURE DE STDIN

→ peut recevoir des données
5. COMPOSABLE

→ sa sortie peut devenir l'entrée d'un autre

À retenir Une CLI bien conçue est **scriptable**. Ce qui peut être écrit dans un script doit l'être dans la CLI. C'est exactement ce qui rend les CLIs utilisables par un agent IA.

GUI vs CLI : ce n'est pas « ancien » contre « moderne »

Tâche	Meilleur outil
Explorer, découvrir, voir d'un coup d'œil	GUI
Répéter 50 fois la même chose	CLI
Stocker en Git, versionner, relire	CLI
Combiner 5 outils	CLI
Apprendre, se former	GUI

Les ingénieurs **utilisent les deux**. La GUI pour comprendre, la CLI pour produire.

Module 2 — Le terminal, le shell, le programme

La confusion à éliminer

Much de gens utilisent ces mots comme synonymes. Ils sont distincts.

COUCHE 1 – LE TERMINAL (l'application)

C'est la FENÊTRE. Elle ne comprend rien aux commandes.

Rôle : afficher des caractères, capter les frappes clavier.

Exemple : Windows Terminal, Terminal macOS, xterm

COUCHE 2 – LE SHELL (l'interpréteur)

C'est le CERVEAU. Il lit ta ligne et décide quoi exécuter.

Rôle : parser, chercher le programme, gérer les pipes.

Exemple : PowerShell (chez toi), bash, zsh, fish

COUCHE 3 – LE PROGRAMME (n8ncli, git, node)

C'est l'OUVRIER. Il fait le vrai travail.

Rôle : tout le reste.

COUCHE 4 – LE SYSTÈME

Fichiers, réseau, permissions, base de données.

L'analogie de la cuisine

TERMINAL = la salle de restaurant (la pièce, la table)
SHELL = le serveur (il prend ta commande, la transmet)
PROGRAMME = le cuisinier (il prépare ton plat)
SORTIE = l'assiette posée devant toi
CODE DE SORTIE = la note « satisfait » ou « à renvoyer en cuisine »

Ce qui se passe quand tu tapes une commande

1. TERMINAL Tu tapes : `n8ncli status`
et appuies sur Entrée
↓
2. TERMINAL Il envoie la chaîne "`n8ncli status`" au shell
↓
3. SHELL Il découpe : le programme s'appelle "`n8ncli`",
les arguments valent ["`status`"]
↓
4. SHELL Cherche "`n8ncli`" dans le PATH
↓ trouve : `C:\Users\ema\AppData\Roaming\npm\n8ncli.ps1`
5. SHELL Démarre un processus et lui passe "`status`" en argument
↓
6. PROGRAMME `n8ncli` démarre (Node.js), lit sa configuration,
se connecte à l'instance `n8n`, travaille
↓
7. PROGRAMME Écrit son résultat sur STDOUT,
puis quitte avec un code (`process.exit(0)`)
↓
8. SHELL Reçoit le code de sortie, le stocke dans `$LASTEXITCODE`,
redonne la main à l'invite de commande

Vérification sur ta machine

```
Get-Command n8ncli
```

```
CommandType ExternalScript  
Source       C:\Users\ema\AppData\Roaming\npm\n8ncli.ps1
```

À retenir Le terminal ne sait **rien**. Le shell sait **où trouver** les choses. Le programme sait **faire** le travail.

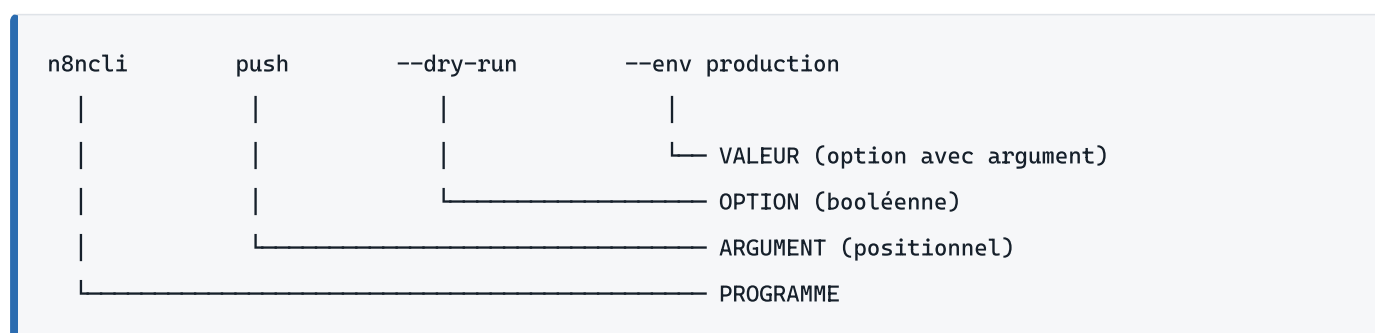
Les différents shells

Shell	Disponible	Note
PowerShell	Windows (ton cas)	Standard Microsoft, objets en sortie
bash	Linux, macOS, Git Bash	Standard Unix
cmd.exe	Windows	Ancien, très limité
zsh / fish	macOS	Plus modernes

Tous font la même chose : `commande + arguments`. La syntaxe des options change légèrement, le principe est identique.

Module 3 — Anatomie d'une commande

La structure universelle



Les 4 types de paramètres

1. LE PROGRAMME

Le nom tapé. Le shell le cherche dans le PATH.

2. L'ARGUMENT POSITIONNEL (SANS TIRET)

```
n8ncli pull "Assistant de Trajet.workflow.ts"
n8ncli pull                                # ← pas d'argument, c'est OK
```

3. L'OPTION BOOLÉENNE (DRAPEAU / FLAG)

```
n8ncli push --dry-run
n8ncli push --all --force --no-cache
```

4. L'OPTION AVEC VALEUR

```
n8ncli --env production push
n8ncli push --config "C:\autre\projet\n8n-cli.json"
```

L'option par défaut et ses variantes

Quand tu vois `--x` et `--no-x`, ce sont souvent la même option.

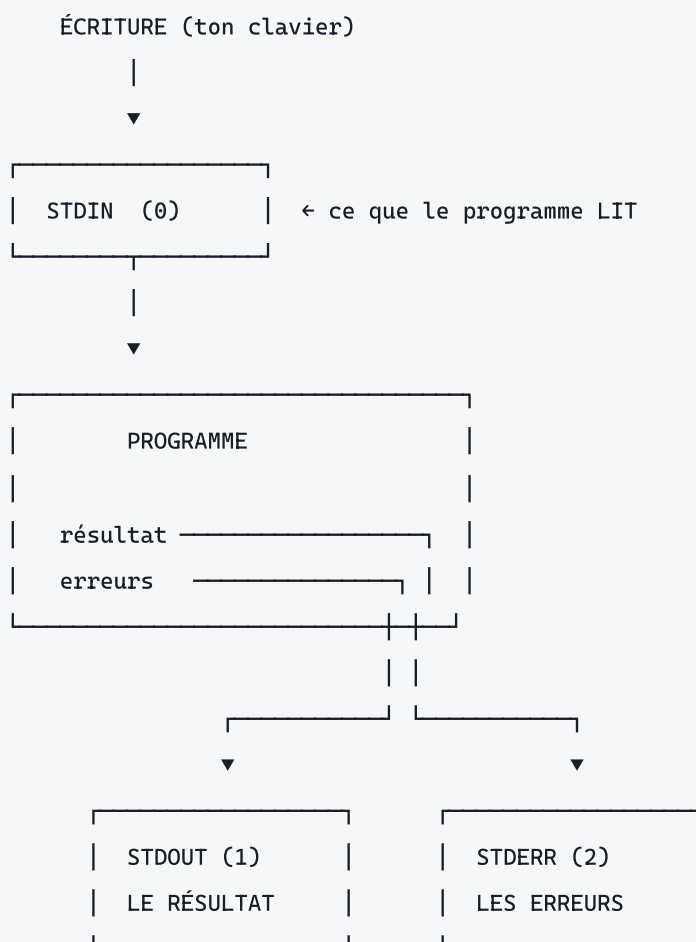
```
n8ncli push --cache      # utilise le cache (rapide, mais peut être périmé)
n8ncli push --no-cache   # ignore le cache (lent, mais fiable)
```

Module 4 — STDIN, STDOUT, STDERR et codes de sortie

C'est le concept le plus important de tout le cours.

Les 3 flux standards

Tout programme qui obéit aux conventions UNIX possède 3 « tuyaux » :



Pourquoi séparer les erreurs du résultat ?

Parce que ça permet de faire :

```
n8ncli status > resultat.txt
```

→ Le fichier contient **seulement** les données, pas les messages d'avertissement. Le fichier reste exploitable.

```
n8ncli status 2> erreurs.txt
```

→ `resultat.txt` n'a que les données, `erreurs.txt` que les erreurs. Séparés.

Les codes de sortie

Un nombre que le programme renvoie en sortant. C'est le **verdict** du programme.

Code	Signification	Exemple
0	Succès	La commande a marché
1	Erreur générique	Connexion impossible, exécution échouée
2	Validation échouée	Le fichier ne respecte pas le schéma
3	Conflit de synchronisation	Modifications concurrentes à résoudre
4 +	Autres (spécifique au programme)	

Vérifier un code de sortie en PowerShell

```
n8ncli validate --lint
if ($LASTEXITCODE -eq 0) {
    Write-Host "OK, on peut pousser"
} else {
    Write-Host "ERREUR code $LASTEXITCODE, on ne pousse pas"
}
```

C'est LA CLEF de l'automatisation

Un humain voit un joli message d'erreur à l'écran. Un script ne voit **que le nombre**.

Programme



```
| Message lisible |  
| (stdout + stderr) |
```

✅ pour un humain

Programme



```
| 3 |  
| (juste un nombre) |
```

✅ pour une machine
→ décision automatique

À retenir Si une CLI n'a pas de code de sortie, elle est mal conçue. C'est le contrat entre le programme et le script.

Module 5 — Pipes et redirections

La redirection (stocker)

```
n8ncli status > sortie.txt      # STDOUT dans le fichier  
n8ncli push 2> erreurs.txt     # STDERR dans le fichier  
n8ncli push > tout.txt 2>&1    # les DEUX dans le même fichier
```

Le pipe (chaîner)

Le symbole `|` envoie la sortie d'un programme vers l'entrée d'un autre.

```
n8ncli status --json | ConvertFrom-Json  
n8ncli status --json | Select-Object -ExpandProperty workflows  
n8ncli status --json | Out-File etat.json -Encoding utf8
```

La puissance de la combinaison

```
# Compter les workflows modifiés
n8ncli status --json | ConvertFrom-Json |
  Select-Object -ExpandProperty modified |
  Measure-Object |
  Select-Object -ExpandProperty Count

# Sauvegarder et afficher
n8ncli status --json | Out-File etat.json
Get-Content etat.json | ConvertFrom-Json | Format-Table
```

Le mode `--json` : le mode « machine »

Toutes les CLIs modernes ont deux modes :

Mode	Pour qui	Exemple de sortie
Humain (par défaut)	Tu, avec tes yeux	✓ 3 workflows modifiés
<code>--json</code>	Programme, script	<code>{"modified": 3, "untracked": 0}</code>

```
n8ncli status          # lisible
n8ncli status --json   # structuré
```

À retenir Si tu veux scripter une commande, ajoute `--json`. Si tu veux la lire, ne l'ajoute pas.

Module 6 — PATH, variables d'environnement

Le PATH

Le PATH est une **liste de dossiers** où le shell cherche les programmes.

Le shell reçoit : "n8ncli"

Il essaie :

```
C:\Windows\n8ncli      → pas trouvé
C:\Program Files\n8ncli → pas trouvé
C:\Users\ema\AppData\Roaming\npm\n8ncli → TROUVÉ ✓
```

Vérification :

```
$env:PATH -split ';' | Where-Object { $_ -like '*npm*' }  
Get-Command n8ncli | Select-Object Source
```

Les variables d'environnement

Des valeurs stockées en mémoire, accessibles par tous les programmes.

```
$env:USERPROFILE          # C:\Users\ema  
$env:N8NCLI_ACCESS_TOKEN  # un secret (si configuré)  
$env:NODE_DEBUG           # active le debug de Node
```

```
# Écrire une variable (session courante uniquement)  
$env:MON_TOKEN = "abc123"  
  
# Écrire une variable (permanente, utilisateur Windows)  
[Environment]::SetEnvironmentVariable("MON_TOKEN", "abc123", "User")
```

Où est installé n8ncli ?

```
npm ls -g --depth=0
```

```
C:\Users\ema\AppData\Roaming\npm  
+-- @workflows-accelerator/n8n-cli@1.2.40  
`-- eas-cli@16.28.0
```

```
npm install -g @workflows-accelerator/n8n-cli    ← installation globale  
          ↑  
le -g (global) le met dans le PATH,  
accessible depuis n'importe quel dossier
```

Les 3 emplacements à connaître

C:\Users\ema\.n8ncli-global.json	← config GLOBALE (secrets)
C:\Users\ema\n8n-projects\	← ton dépôt (le code)
C:\Users\ema\AppData\Roaming\npm\	← les programmes installés

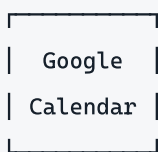
PARTIE 2 — n8n (le serveur)

Module 7 — Qu'est-ce que n8n

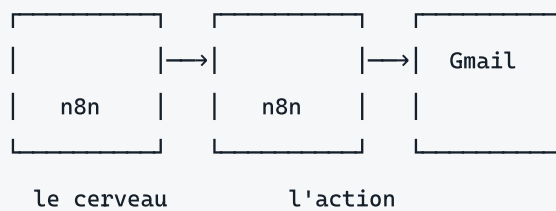
Définition

n8n est une **plateforme d'automatisation** : tu connectes des applications entre elles pour qu'elles travaillent ensemble sans intervention humaine.

Sans n8n :



Avec n8n :



Le concept central

TRIGGER (déclencheur) → ACTION (action)

"QUAND"

"QUOI"

"À 9h30 du lundi" + "envoie un email"

L'analogie du robot

n8n est un **robot d'automatisation** :

- **Déclencheur** = le bouton sur lequel on appuie, ou l'heure à laquelle il se réveille
- **Nœud** = une action que le robot sait faire
- **Connexion** = le fil qui relie les actions dans l'ordre
- **Exécution** = une fois que le robot a fait le travail, avec la trace de ce qu'il a fait

n8n vs Zapier / Make

	n8n	Zapier / Make
Hébergement	Chez toi ou n8n Cloud	Chez eux, uniquement
Code	Oui (nœud Code, JavaScript/Python)	Non
Workflows en base de données	Oui	Non
Versionnement Git	Possible (avec n8n CLI)	Non
Coût	Gratuit en self-hosted, Cloud payant	Abonnement par tâche
Personnalisation	Totale	Limitée

Le gros avantage de n8n : tu possèdes tes données et ton code.

Module 8 — Les concepts fondamentaux

Workflow (flux)

Un workflow = un ensemble de nœuds reliés, qui fait une tâche.

[Déclencheur] → [Nœud 1] → [Nœud 2] → [Nœud 3]

QUANDCE FAITPUIS

ET ALORS

Nœud (node)

C'est **une unité de travail** : un appel à une application, ou une transformation de données.

[Google Sheets] [HTTP Request] [Code] [Gmail] [IF]

Lire desAppeler uneExécuterEnvoyerCondition

donnéesAPI externe

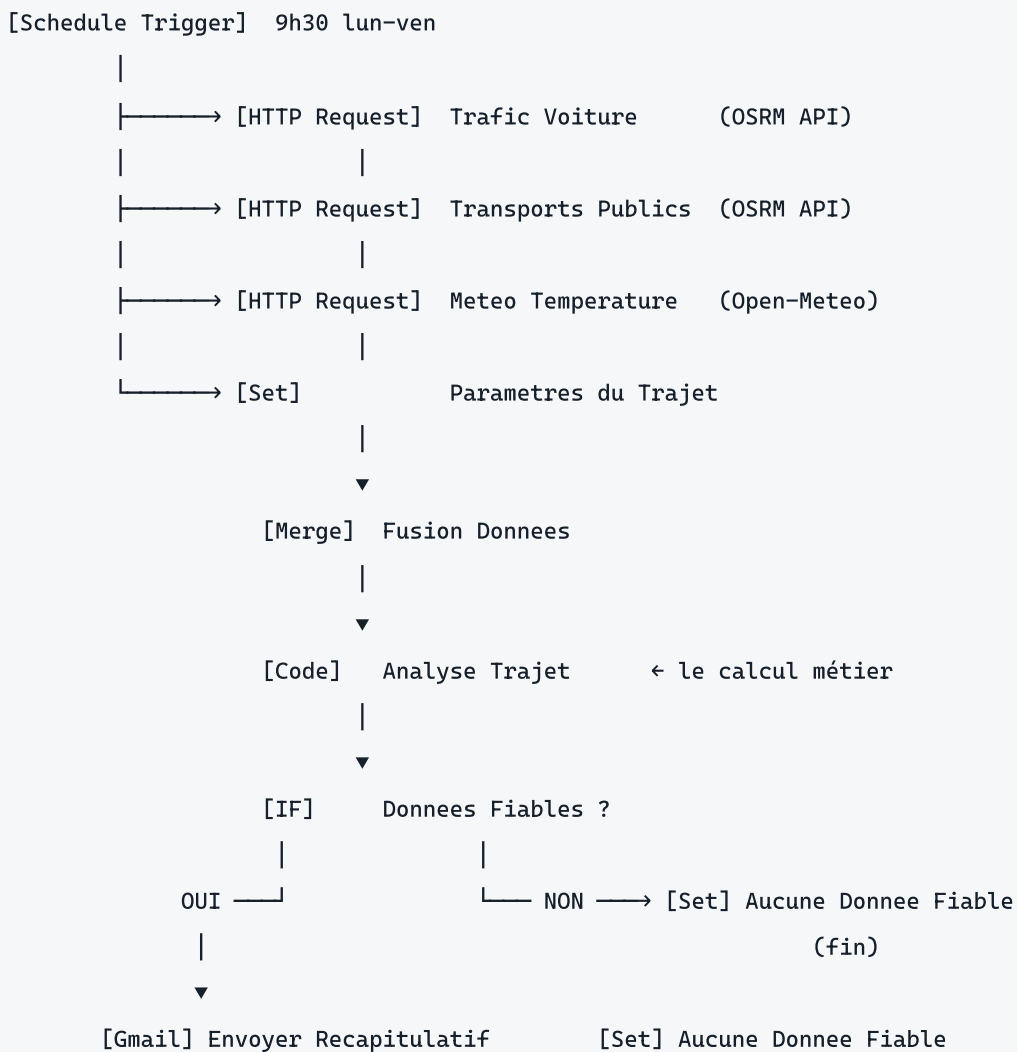
du JSun email

Les 4 grandes familles de nœuds

1. TRIGGERS (déclencheurs) → le workflow démarre
Schedule Trigger, Webhook, Manual, Email, Sub-workflow
2. ACTIONS → ils font quelque chose
Google Sheets, Gmail, Slack, HTTP Request, Postgres
3. TRANSFORMATIONS → ils transforment les données
Code, Edit Fields (Set), Item Lists, Aggregate
4. LOGIQUE → ils décident
IF, Switch, Filter, Merge

Exemple : ton vrai workflow

n8n/workflows/Assistant de Trajet - Recapitulatif Matinal.workflow.ts



Connexions

```
Nœud A —————> Nœud B      relation simple
Nœud A —┬───> Nœud B      un nœud, plusieurs sorties (parallèle)
        └───> Nœud C
Nœud A —┬───> Nœud B      plusieurs nœuds fusionnent
Nœud D —┴──┘
```

Exécution (execution)

Une exécution = une fois que le workflow a tourné.

```
Exécution #1 — 01/10/2026 09:30:02 → SUCCÈS (1.2s)
Exécution #2 — 02/10/2026 09:30:01 → SUCCÈS (1.1s)
Exécution #3 — 03/10/2026 09:30:00 → ÉCHEC (erreur API OSRM)
```

Chaque exécution est **enregistrée** dans la base de données. C'est ton historique, ton journal de bord.

Module 9 — Le modèle de données

C'est la partie la plus importante à comprendre, sinon rien ne fait sens.

Tout est un item (un objet JSON)

Le résultat d'un nœud = une LISTE d'items

Chaque item = un objet JSON

```
[
  { "json": { "id": 1, "name": "Alice" } },    ← item 1
  { "json": { "id": 2, "name": "Bob" } }      ← item 2
]
```

Exemple concret

[Google Sheets] lit une feuille :

```
[
  { "json": { "email": "alice@mail.com", "ville": "Paris" } },
  { "json": { "email": "bob@mail.com", "ville": "Lyon" } },
  { "json": { "email": "carol@mail.com", "ville": "Marseille" } }
]
```

[IF] filtre sur ville = "Paris" :

```
[
  { "json": { "email": "alice@mail.com", "ville": "Paris" } }
]
```

[Gmail] envoie un mail à chaque item restant :

→ 1 email envoyé

`$json` = l'item courant

Dans un nœud, `$json` = l'item qu'on est en train de traiter

`{{ $json.email }}` → "alice@mail.com"

`{{ $json.ville }}` → "Paris"

`$( )` = aller chercher le résultat d'un autre nœud

Dans ton workflow :

`{{ $("Parametres du Trajet").first().json.emailDestinataire }}`

				└─ la valeur du champ
			└─	le PREMIER item
	└─			le nœud par son NOM
└─				la fonction d'accès

`expr()` : la fonction du SDK TypeScript

```
// Dans un .workflow.ts :  
url: expr('{{ $json.email }}')  
//           |_____| l'expression, entre guillemets simples  
//  
// Dans l'éditeur visuel n8n :  
// = {{ $json.email }} → deux accolades, sans guillemets
```

Attention — le piège n°1

- **Éditeur n8n** : `{{ }}` dans un champ texte
- **SDK TypeScript** : `expr('...')` avec guillemets simples

Les deux sont différents. Ne pas confondre.

Le modèle « liste d'items » explique les erreurs

Erreur fréquente : "expected 1 item but received 3"

Pourquoi ? Tu fais 1 action (1 email) mais le nœud précédent a produit 3 items. n8n essaie de faire 1 action sur 3 données.

Solutions :

- "Run Once for All Items" → traiter la liste d'un coup
- "Run Once for Each Item" → faire l'action 3 fois
- un nœud Aggregate pour grouper

Module 10 — Les credentials

Définition

Un credential = **un secret stocké côté serveur n8n**, jamais dans le code.

Code du workflow :

```
credentials: { gmailOAuth2: newCredential('Gmail account', '80u9cDgullboEIRf') }  
                        |_____|  
                        une RÉFÉRENCE, pas le secret
```

Côté n8n (serveur) : l'ID 80u9cDgullboEIRf pointe vers le vrai refresh token

La règle d'or

||

Le secret vit EXCLUSIVEMENT dans la base de données n8n.

||

||

Le workflow ne contient qu'un identifiant.

||

||

Ce secret ne doit JAMAIS apparaître dans un fichier.

||

Pourquoi c'est bien conçu

	Si le secret est dans le code	Si le secret est côté serveur (n8n)
Qui voit le secret ?	Tous qui voient le fichier	Seulement n8n
Un collègue voit le fichier ?	❌ Il a ton mot de passe Gmail	✅ Il voit juste un ID
Push sur Git ?	💀 FUITE	✅ Pas de problème
Rotation du secret	Modifier le code	Modifier dans l'UI n8n

Le piège : les nœuds Code

Un nœud Code est du **vrai JavaScript qui tourne**. Si tu y écris une clé en dur, elle finit dans Git.

```
// JAMAIS
const apiKey = "sk-proj-abc123xyz";

// Si tu dois vraiment le faire (à éviter)
const apiKey = $env.API_KEY;
```

Module 11 — Le cycle de vie d'un workflow

Le concept le plus important de la partie serveur.



Les implications

État	Le workflow existe ?	Déclencheurs ?	Historique ?
Inactif (non publié)	Oui, dans l'éditeur	Non	Non
Actif (publié)	Oui, dans l'éditeur	Oui	Oui

Les trois niveaux de contrôle

n8ncli exec <file> --mode manual	exécution manuelle
↓	(pas d'effet si inactif)
n8ncli test <file>	test local avec données simulées
↓	(jamais d'effet réel)
n8ncli publish <file>	ACTIVE le workflow
	= effets réels dans le monde

À retenir Publier, c'est faire sortir une automatisation dans la réalité. C'est le moment le plus dangereux du cycle de vie.

Module 12 — Où sont stockées les données

n8n est une application **Node.js** qui utilise une **base de données**.

Les tables principales (PostgreSQL)

<code>workflow_entity</code>	-- les workflows (le JSON de chaque workflow)
<code>execution_entity</code>	-- les exécutions (historique)
<code>credentials_entity</code>	-- LES SECRETS
<code>webhook_entity</code>	-- les routes webhooks actives
<code>execution_data</code>	-- les données de chaque exécution
<code>user_entity</code>	-- les comptes utilisateurs
<code>project</code>	-- les projets (regroupement de workflows)
<code>folder</code>	-- les dossiers

Les implications pour la sécurité

1. Un accès PostgreSQL = un accès TOTAL
→ on court-circuite toute la validation applicative
2. Les secrets sont dans `credentials_entity`
→ n'importe qui qui y a accès peut lire les credentials
3. `execution_data` contient les DONNÉES de tes clients
→ l'historique est une base de connaissances de ton entreprise

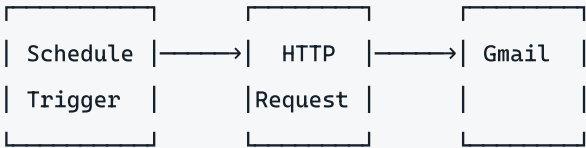
Conclusion : la base de données est la cible n°1.

PARTIE 3 — n8n CLI

Module 13 — Le problème que ça résout

Le problème : n8n classique stocke du JSON illisible

Ce que tu vois dans l'éditeur n8n (graphique) :



Ce qui est vraiment stocké en base :

```
[{ "nodes": [
  {
    "parameters": {...180 lignes...},
    "name": "Schedule Trigger",
    "position": [250, 300],
    "typeVersion": 1.4,
    ...},
  {
    "parameters": {"url": "https://..."},
    "name": "HTTP Request",
    "position": [450, 300],
    ...}
],
"connections": {...},
"settings": {...},
"id": "rqDxQgeCC393cbRI",
"active": false
}]
```

Les 6 problèmes du JSON

Problème	Conséquence
Pas de diff	Impossible de savoir ce qui a changé
Pas de code review	On ne peut pas relire un JSON de 800 lignes
Pas de branches Git	Impossible de travailler à deux
Merge cassant	Deux personnes modifient → conflit inextricable
Pas d'historique lisible	On ne sait pas <i>pourquoi</i> ça a changé
Pas de rollback fiable	On ne peut pas revenir en arrière proprement

La solution de n8n CLI

DANS LE JSON :

```
[{ "parameters": {  
  "rule": {  
    "interval": [{  
      "field": "cronExpression",  
      "expression": "30 9 * * 1-5"  
    }]  
  },  
  "name": "Schedule  
Trigger", ...
```

DANS LE SDK :

```
node({  
  type: 'n8n-nodes-base.scheduleTrigger',  
  version: 1.4,  
  config: {  
    name: 'Declenchement Matinal',  
    parameters: {  
      rule: { interval: [{ field:  
        field: 'cronExpression',  
        expression: '30 9 * * 1-5' }] }  
    }  
  }  
})
```

On obtient :

- ✓ Un diff ligne par ligne
- ✓ Une relecture de code possible
- ✓ Des branches Git
- ✓ Des merges lisibles
- ✓ Un historique
- ✓ Un rollback par commit

Module 14 — Workflow-as-Code : anatomie d'un fichier

Regardons ton **vrai fichier**, ligne par ligne.

Exemple réel

```
n8n/workflows/Assistant de Trajet - Recapitulatif Matinal.workflow.ts
```

Étape 1 : un nœud est une `const`

```
// lignes 1 à 5
const declenchement_Matinal = trigger({
  type: 'n8n-nodes-base.scheduleTrigger',
  version: 1.4,
  config: {
    name: 'Declenchement Matinal',
    parameters: {
      rule: { interval: [{ field: 'cronExpression', expression: '30 9 * * 1-5' }] }
    },
    position: [0, 96],
    notes: 'Declenche le recapitulatif du lundi au vendredi a 11h30...',
    notesInFlow: true
  }
});
```

Décryptage du format :

<code>const nom_variable = trigger({</code>	← la fonction (trigger/node/merge/switchCase)
<code> type: 'n8n-nodes-base.xxx',</code>	← le TYPE du nœud
<code> version: 1.4,</code>	← la VERSION du nœud
<code> config: {</code>	← tout ce qui décrit le nœud
<code> name: '...',</code>	← son nom
<code> parameters: { ... },</code>	← sa configuration
<code> position: [x, y],</code>	← sa position dans le canvas
<code> notes: '...',</code>	← sa documentation
<code> notesInFlow: true</code>	← afficher la note sur le canvas
<code> }</code>	
<code>});</code>	

Les 4 factory functions

<code>trigger({...})</code>	← un déclencheur (début du workflow)
<code>node({...})</code>	← un nœud classique
<code>merge({...})</code>	← un nœud de fusion
<code>switchCase({...})</code>	← un nœud de routage

Exemple réel 2 : un nœud HTTP Request

```
// lignes 18 à 22
const trafic_Voiture = node({
  type: 'n8n-nodes-base.httpRequest',
  version: 4.5,
  config: {
    name: 'Trafic Voiture',
    parameters: {
      method: 'GET',
      url: '{{ "https://router.project-osrm.org/route/v1/driving/" + $json.lonDepart + "..."} }',
      options: {
        response: {
          response: { responseFormat: 'json', neverError: true, timeout: 20000 }
        }
      }
    },
    position: [496, 224],
    notes: 'Duree de trajet voiture domicile vers gare via OSRM, gratuit et sans cle...'
  }
});
```

Points importants :

- `neverError: true` → si l'API échoue, le workflow continue (le nœud suivant vérifie)
- `timeout: 20000` → 20 secondes maximum
- `notes` → la documentation du nœud, vit dans le code

Exemple réel 3 : les credentials

```
// ligne 51
const envoyer_Recapitulatif = node({
  type: 'n8n-nodes-base.gmail',
  version: 2.2,
  config: {
    name: 'Envoyer Recapitulatif',
    parameters: {
      sendTo: expr('{{ $("Parametres du Trajet").first().json.emailDestinataire }}'),
      subject: expr('{{ "Trajet du matin - " + ($json.perturbation ? "perturbation detectee" :
"trajet normal") }}'),
      message: expr('{{ $json.message }}')
    },
    credentials: { gmailOAuth2: newCredential('Gmail account', '80u9cDgullboEIRf') },
    ...
  }
});
```

```
// newCredential(NOM_AFFICHÉ, ID_DU_SECRET)
//
//           ↓
//      "80u9cDgullboEIRf" → un ID, PAS le token OAuth
```

Point clé : cet ID n'est pas un secret. Il ne donne accès à rien sans l'instance n8n. Mais il révèle le **nom** du credential (`Gmail account`), ce qui est une petite fuite d'information.

Exemple réel 4 : un nœud Code

```
// lignes 36 à 40 (début)
const analyse_Trajet = node({
  type: 'n8n-nodes-base.code',
  version: 2,
  config: {
    name: 'Analyse Trajet',
    parameters: {
      mode: 'runOnceForAllItems',
      language: 'javascript',
      jsCode: `const cfg = $('Parametres du Trajet').first().json;
               const carBody = $('Trafic Voiture').first().json || {};
               ...
               const perturbation = retardTotal >= seuil;
               const risque = retardTotal === 0 ? 'faible' : ...;`
    }
  }
});
```

`runOnceForAllItems` = traiter **tous** les items en une fois (pas une boucle).

Exemple réel 5 : l'assemblage

```
// ligne 60
const wf = workflow('rqDxQgeCC393cbRI', 'Assistant de Trajet - Recapitulatif Matinal', {
  executionOrder: 'v1',
  availableInMCP: true,
  binaryMode: 'separate'
});
```

```
workflow(ID, NOM, OPTIONS)
//      |      |
//      |      └─ le nom affiché dans n8n
//      └───────── l'ID dans la base n8n (identifiant serveur)
```

Les options du workflow

Option	Signification
<code>description</code>	Obligatoire par les standards du projet
<code>active</code>	<code>true</code> = activé sur l'instance
<code>availableInMCP</code>	Autorise l'accès MCP à ce workflow
<code>executionOrder</code>	<code>v1</code>
<code>binaryMode</code>	<code>separate</code> = fichiers binaires séparés
<code>settings</code>	options d'exécution

La syntaxe de chaînage

```
const wf = workflow('id', 'Nom', {});

export default wf
  .add(declenchement_Matinal)    // ajoute au graphe
  .add(trafic_Voiture)
  .to(declenchement_Matinal, trafic_Voiture)    // A → B
  .connect('A', 'B')              // syntaxe alternative
  ;
```

Les méthodes de connexion :

Méthode	Usage
<code>.to(A, B)</code>	A sort vers B
<code>.add(X)</code>	ajoute X au graphe sans le connecter
<code>.connect('A', 'B')</code>	connexion par noms
<code>.output(1)</code>	sortie n°1 (nœuds multi-sorties)
<code>.onCase(0, X)</code>	branche 0 d'un Switch
<code>.to([X, Y])</code>	A vers X et Y en parallèle
<code>.input(0)</code>	entrée 0 d'un Merge

La structure finale d'un fichier

```
// =====  
// 1. IMPORTATIONS  
// =====  
import { workflow, node, trigger, merge } from '@n8n/workflow-sdk';  
  
// =====  
// 2. DÉFINITION DES NŒUDS (une const par nœud)  
// =====  
const a = trigger({ ... });  
const b = node({ ... });  
const c = node({ ... });  
  
// =====  
// 3. DÉFINITION DU WORKFLOW  
// =====  
const wf = workflow('id', 'Nom', { ... });  
  
// =====  
// 4. ASSEMBLAGE (le graphe)  
// =====  
export default wf  
  .add(a)  
  .to(a, b)  
  .to(b, c);
```

À retenir : un `.workflow.ts` n'est pas un fichier de configuration. C'est du **vrai code** lisible, versionnable et relisible.

Module 15 — Architecture : les 4 couches de configuration

COUCHE 1 – GLOBALE (hors dépôt, secrète)

```
| C:\Users\ema\.n8ncli-global.json |  
|  
| { "environments": { |  
|   "development": { |  
|     "instanceUrl" : "https://ema1999.app.n8n.cloud", |  
|     "mcpCommand"  : "npx -y n8n-mcp", |  
|     "accessToken"  : "<272 caractères, EN CLAIR>" |  
|   } } } |
```

↑ Contient les SECRETS. Jamais dans Git.

COUCHE 2 – PROJET (dans le dépôt, versionnée)

```
| n8n/config/n8n-cli.json      ← env, projectId, dossiers |  
| n8n/config/n8n-standards.json ← règles de nommage |  
| n8n/config/n8n-layout.json   ← distances de mise en page |
```

↑ Config PARTAGÉE. Pas de secret. Va dans Git.

COUCHE 3 – ÉTAT LOCAL (dans le dépôt, gitignoré)

```
| n8n/config/sync-state.json      ← empreinte du sync |  
| n8n/config/unconfigured-credentials.json ← credentials non |  
|                                     résolus localement |  
| n8n/config/workflow-folders.json ← structure |  
| n8n/config/cache/              ← cache résolution |
```

↑ Interne, reproductible. Jamais dans Git.

COUCHE 4 – DONNÉES (le code métier)

```
| n8n/workflows/**/*.workflow.ts ← LES WORKFLOWS |  
| n8n/references/                ← exemples de référence |
```

↑ Le code. C'est ce qu'on versionne et relit.

Ton `n8n-cli.json` réel

```
{
  "env": "development",
  "localDir": "n8n",
  "projectId": "9czvyKNQTLvUqkp7",
  "projectName": "emilie <efernandes@eugeniaschool.com>",
  "references": [
    { "name": "Workflow Examples Ref", "builtin": "examples" }
  ]
}
```

Champ	Signification
<code>env</code>	quel environnement utiliser (<code>development</code>)
<code>localDir</code>	où sont les workflows (<code>n8n/</code>)
<code>projectId</code>	quel projet n8n cibler
<code>projectName</code>	son nom
<code>references</code>	les exemples à charger

Ton `.gitignore` réel

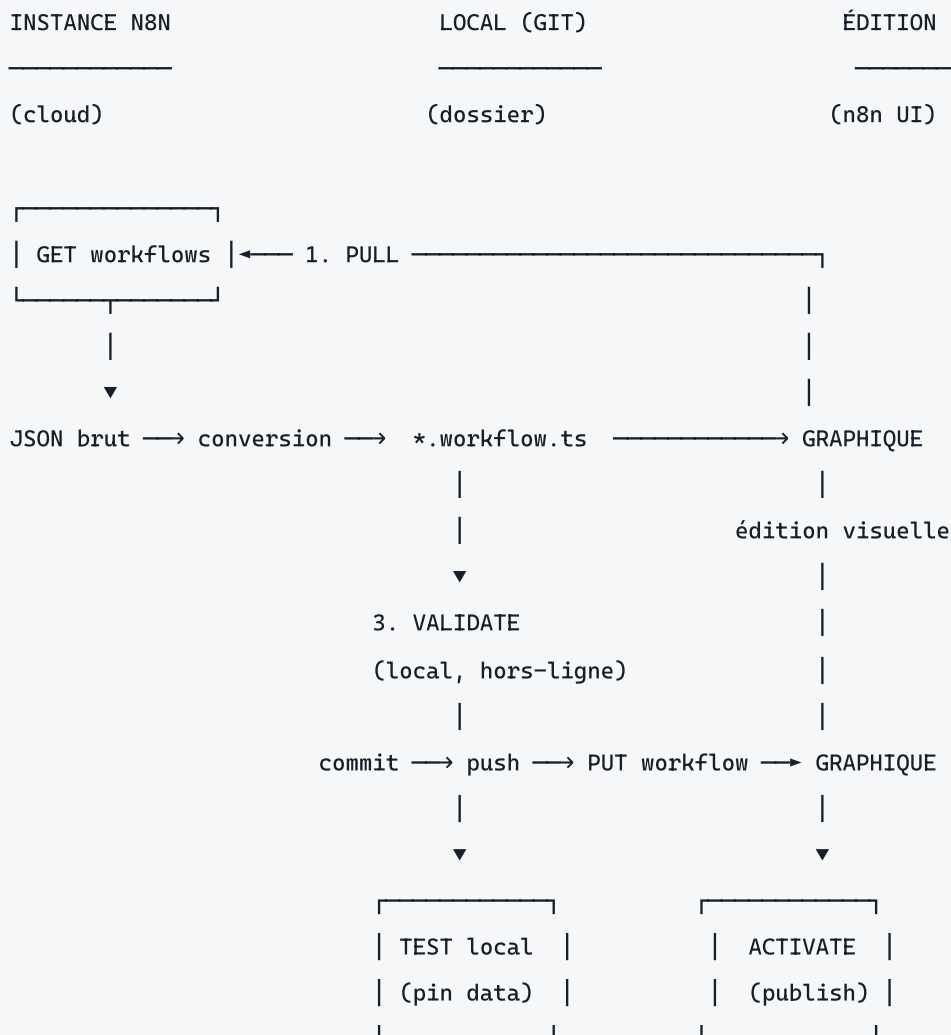
```
.env
n8n/config/sync-state.json
n8n/config/unconfigured-credentials.json
n8n/config/workflow-folders.json
n8n/references/
```

Lecture : « ces fichiers sont importants pour mon outil, mais ils n'ont rien à faire dans Git ».

- `.env` → secrets locaux
 - `sync-state.json` → état interne, se régénère
 - `unconfigured-credentials.json` → **contient des références de credentials**
 - `references/` → régénéré par `n8ncli pull`
-

Module 16 — Le cycle de vie du développement

Le flux complet



Étape 1 : PULL — récupérer l'état distant

```
n8ncli pull
```

Que fait-il ?

1. Se connecte à l'instance n8n (MCP ici)
2. Récupère les workflows du projet
3. Convertit JSON → TypeScript
4. Écrit les fichiers `.workflow.ts`
5. Renomme chaque fichier selon le NOM DISTANT du workflow
6. Met à jour `sync-state.json`

Le piège du renommage : `pull` utilise le nom distant comme nom de fichier.

```
Workflow distant nommé "Assistant de Trajet - Recapitulatif Matinal"
→ fichier local : "Assistant de Trajet - Recapitulatif Matinal.workflow.ts"

Si tu l'avais appelé "trajet-matin.workflow.ts", le pull le renomme !
```

Conséquence : ne nomme pas tes fichiers à la main. Laisse le pull décider.

```
n8ncli pull --dry-run      # voir ce que ça ferait, sans écrire
n8ncli pull --force       # écraser les modifications locales
```

Étape 2 : DÉVELOPPER — écrire le code

```
# Voir la documentation du SDK
n8ncli sdk all

# Chercher un nœud
n8ncli nodes search gmail

# Lire sa documentation et ses exemples copiables
n8ncli nodes doc n8n-nodes-base.gmail:message:send

# Voir les types TypeScript exacts
n8ncli nodes types n8n-nodes-base.gmail:message:send
```

Ne devine jamais les paramètres d'un nœud. Demande au CLI. C'est le CLI qui a le schéma, pas toi.

Les exemples de référence disponibles dans ton dépôt :

```
n8n/references/workflow_examples_ref/
├─ Sub Workflows/           Child Workflow, Caller Workflow
├─ Logic/                   Logic Template
├─ Complex Patterns/        Layout Test, Data Orchestration and RAG
├─ Code/                    Code Template
├─ API Endpoints/           API Webhook Template
└─ AI Prompts/              AI Prompt Template
```

Étape 3 : VALIDATE — vérifier localement

```
n8ncli validate --lint
```

Vérifie :

- ✓ syntaxe TypeScript
- ✓ conformité au schéma n8n
- ✓ versions des nœuds (typeVersion)
- ✓ références aux nœuds dans les expressions
- ✓ standards de nommage du projet

Aucun réseau, aucune écriture. C'est un contrôle qualité gratuit.

```
n8ncli validate --fix           # corrige les versions automatiquement
n8ncli validate --upgrade-nodes # passe les nœuds en dernière version
n8ncli validate --only-modified # ne vérifie que ce qui a changé
```

Étape 4 : PUSH — déployer

```
n8ncli status      # toujours AVANT. Qu'est-ce qui a changé ?
n8ncli diff --semantic # quelles sont les différences ?
n8ncli push --dry-run  # que va-t-il se passer ?
n8ncli push         # envoie
```

push fait :

1. Lit les `.workflow.ts` locaux
2. Les convertit en JSON
3. Compare avec `sync-state.json` → liste les changements
4. Envoie uniquement ce qui a changé
5. Met à jour `sync-state.json`

Option	Effet
<code>--dry-run</code>	simule, n'écrit rien
<code>--force</code>	écrase sans demander
<code>--hard</code>	réinitialisation complète
<code>--all</code>	tout traiter
<code>--no-cache</code>	force un build propre
<code>--prune</code>	purge les doublons en base

Étape 5 : TESTER

```
n8ncli test "Assistant de Trajet.workflow.ts"
n8ncli test <file> --pin-data mon-pin.json
```

Un **test local** avec des **données simulées** (pin data) : tu ne touches pas aux vraies API.

```
{
  "Trafic Voiture": [
    { "json": { "code": "Ok", "routes": [{ "duration": 720 }] } }
  ]
}
```

Étape 6 : PUBLIER

```
n8ncli publish "Assistant de Trajet.workflow.ts"    # ACTIVE
n8ncli unpublish "Assistant de Trajet.workflow.ts"  # désactive
```

L'alternative : la boucle live

```
n8ncli live --interval 30 --foreground
```

Synchronisation automatique continue (tire et pousse toutes les 30 secondes).

```
n8ncli live --status    # état du démon
n8ncli live --stop      # l'arrêter
```

Module 17 — Le catalogue de commandes

1. Configuration et découverte

```
n8ncli init                # initialise le workspace
n8ncli init --reset        # réinitialise standards et cache
n8ncli env list            # liste les environnements
n8ncli env test development # teste API, MCP et BDD
n8ncli env edit development # configure un environnement
n8ncli projects            # liste les projets n8n
```

```
n8ncli folders list --tree          # arborescence des dossiers
n8ncli folders create "Mon Dossier"
n8ncli folders move <workflow> <folder>
n8ncli folders delete <folder> --dry-run
n8ncli folders set-parent <folder> <parent>
```

2. Synchronisation

```
n8ncli pull                        # distant → local
n8ncli push                        # local → distant
n8ncli status                      # quoi a changé ?
n8ncli diff --semantic             # différences détaillées
```

3. Validation et standards

```
n8ncli validate                   # vérifie le schéma
n8ncli validate --lint            # + les standards du projet
n8ncli validate --fix             # corrige automatiquement
n8ncli validate --upgrade-nodes   # passe en dernière version
n8ncli lint                       # standards uniquement
n8ncli lint --fix                 # corrige les noms dupliqués
n8ncli standards                  # gère n8n-standards.json
```

4. Exécution et debug

```
n8ncli exec <file>                # exécute
n8ncli exec <file> --mode manual
n8ncli exec <file> --mode production # effets réels
n8ncli exec <file> --input '{"email":"x"}' # données d'entrée

n8ncli test <file>                # test local
n8ncli test <file> --pin-data <fichier> # avec données mockées

n8ncli execution <file> <exec-id>   # détails d'une exécution
n8ncli execution inspect <exec-id>  # stack trace détaillée
n8ncli logs <file> --last-failed    # dernières erreurs
n8ncli debug <exec-id>              # erreur d'un exec précis
```

```
n8ncli webhooks verify             # audit des webhooks vs workflows actifs
```

5. Nœuds et SDK (pour écrire du code)

```
n8ncli nodes search gmail slack    # recherche de nœuds
n8ncli nodes types <nodeId>        # les types TypeScript exacts
n8ncli nodes doc <nodeId>          # doc interactive + exemples copiables
n8ncli sdk                          # les guidelines du SDK
n8ncli sdk all                      # toute la référence
n8ncli sdk expressions             # une section spécifique
```

6. Mise en page

```
n8ncli layout                      # auto-positionne avec Dagre
n8ncli layout --nodesep 150 --ranksep 200
n8ncli layout --dry-run
```

7. Base de données

```
n8ncli datatables list
n8ncli datatables query executions --limit 10
n8ncli datatables query ma_table --filter "status='active'"
n8ncli datatables update ma_table --data '{"col":1}' --filter "id=42"
```

8. Aide et options globales

```
n8ncli --help                      # aide globale
n8ncli push --help                 # aide sur push
n8ncli --version                   # 1.2.40
n8ncli --verbose <cmd>             # logs détaillés
n8ncli --json <cmd>               # sortie structurée
n8ncli --env <name> <cmd>         # environnement spécifique
n8ncli --config <chemin> <cmd>    # autre fichier de configuration
```

Module 18 — Codes de sortie et automatisation

Code	Signification	Que faire
0	Succès	Continuer
1	Erreur d'exécution / connexion	Vérifier la config, réessayer
2	Validation / standards échoués	Corriger le code, ne pas pousser
3	Conflit de synchronisation	Résolution manuelle requise

Les 4 lignes à avoir en tête

```
# 1. Vérifier l'état
n8ncli status

# 2. Valider localement (gratuit, hors-ligne)
n8ncli validate --lint

# 3. Voir ce qui va partir
n8ncli push --dry-run

# 4. Pousser
n8ncli push
```

Le script CI idéal

```
# Vérifier la syntaxe et les standards
n8ncli validate --lint --only-modified --fail-on-warnings

if ($LASTEXITCODE -ne 0) {
    Write-Host "ECHEC DE VALIDATION ($LASTEXITCODE)"
    exit 1
}

Write-Host "Validation OK"
exit 0
```

--fail-on-warnings est la clé : sans lui, un avertissement ne bloque rien.

Module 19 — Les standards imposés

Ton `n8n-standards.json` impose des règles. Elles sont **vérifiées automatiquement**.

Le tableau des règles

Élément	Règle	Exemple valide	Exemple invalide
Dossier	<code>^[A-Z][a-zA-Z0-9\s()-]*\$</code>	Projects , Utils	projects , mes_dossiers
Workflow	<code>^[A-Z][a-zA-Z0-9\s()-]*\$</code>	Daily Backup	daily_backup
Noms interdits	liste noire	Rapport Hebdo	My workflow , New workflow , Untitled workflow , Workflow 1
Description	obligatoire, non vide	'Sauvegarde quotidienne'	(vide)
Nœud	<code>^[A-Z][a-zA-Z0-9\s()\-:/*]*\$</code>	Get Users , POST /submit-lead	get_users , node1
Nœuds Code	notes obligatoires	notes: 'Somme les valeurs'	(pas de notes)
Doublons	format <code>(n)</code>	Send Email (1) , Send Email (2)	Send Email , Send Email copy
Variables Set	<code>camelCase</code>	adresseDepart , lonGare	adresse_depart , LON_GARE
Langue	anglais obligatoire	'Fetch user data'	'Recuperer les donnees'

Les couleurs des notes

Couleur	Code	Signification
Rouge	1	Zone à corriger
Bleu	2	Spécification / comportement attendu
Vert	3	Idée / amélioration future
Violet	4	À relire par l'équipe

Les termes tolérés

Certains termes techniques sont tolérés (noms de variables) :

```
sub-workflows, itemId, sub-workflow, defineBelow, executeOnce,  
high-value, Category-based, low-value, Re-converges, Metadata,  
pgvector, PostgreSQL, retrieval-augmented, LLM, SaaS, backend, LangChain
```

Les règles d'ignorance

Pour empêcher qu'un fichier soit synchronisé, validé ou linté :

```
// n8ncli-ignore      dans les 10 premières lignes  
// n8ncli-push-ignore empêche uniquement le push
```

PARTIE 4 — SÉCURITÉ

Module 20 — Le modèle de menace

Avant de sécuriser, il faut savoir **ce qu'on protège** et **contre qui**.

Ce qu'on protège

1. LES SECRETS

→ tokens API, mots de passe OAuth, clés de service
2. LES DONNÉES

→ emails clients, données métier, données personnelles
3. LA PRODUCTION

→ le fonctionnement des workflows en cours
4. LA TRAÇABILITÉ

→ qui a modifié quoi, et quand

Les menaces

Menace	Description	Impact
Secret dans Git	Une clé committée par erreur	Critique — projet entier compromis
Token volé	Le fichier de config est lu	Critique — accès à toute l'instance
Écriture non intentionnelle	<code>push</code> sans vérifier	Élevé — production cassée
Publication accidentelle	<code>publish</code> par erreur	Élevé — déclencheurs réels
Accès base direct	<code>--db-url</code>	Élevé — contournement applicatif
Fuite de logs	<code>--include-data</code> avec données réelles	Moyen — données personnelles exposées
Dépendance malveillante	<code>npx -y</code> téléchargé à chaque fois	Moyen — supply chain

Module 21 — Où sont les secrets

La carte complète de ton installation

1. TOKEN MCP / API

C:\Users\ema\.n8ncli-global.json

environments.development.accessToken

→ 272 caractères, en CLAIR

✓ HORS du dépôt Git

X En clair sur le disque

X Lu par tout processus de ton compte Windows

⚠ Sauvegardé par OneDrive si le dossier est synchronisé

2. SECRETS N8N (Gmail OAuth, API keys)

Base de données n8n, table credentials_entity

✓ JAMAIS dans un fichier de workflow

✓ Jamais dans Git

⚠ Accessibles si quelqu'un a la BDD ou le token MCP

3. .env

./.env ou n8n/config/.env

→ gitignoré ✓

4. RÉFÉRENCES DE CREDENTIALS

Dans les .workflow.ts :

```
newCredential('Gmail account', '80u9cDgullboEIRf')
```

✓ Ce n'est PAS le secret, juste un ID

⚠ Mais ça révèle le NOM du credential

→ fuite d'information mineure

```
| 5. DONNÉES D'EXÉCUTION (le vrai danger) |
| n8ncli execution --include-data          |
| n8ncli test --pin-data                  |
|                                         |
| X Peut contenir : emails, tokens, données clients |
| ⚠ Ne JAMAIS commiter un pin data réel      |
```

Le principe fondamental

```
|| UN SECRET N'APPARAÎT JAMAIS DANS UN FICHIER VERSIONNÉ. ||
||                                                         ||
|| Dans le code → une RÉFÉRENCE (un ID)                    ||
|| Chez n8n     → le VRAI secret                          ||
```

Auditer avant de commiter

```
# Chercher des motifs de secrets dans les workflows
git grep -nE "(sk-|api[_-]?key|password|Bearer |-----BEGIN)" -- "*.workflow.ts"

# Vérifier ce qui est sur le point d'être commité
git status
git diff --cached

# Vérifier l'historique
git log -p -S "sk-" --all
```

Durcir le fichier global

```
# Vérifier les permissions
icaccls "$env:USERPROFILE\.n8ncli-global.json"

# Alternative : sortir le token d'un fichier en clair
$env:N8NCLI_ACCESS_TOKEN = "..."
```

Module 22 — Les 6 vrais points de risque

Risque 1 : `--db-url` — l'accès direct à la base

```
n8ncli logs <workflow> --db-url "postgresql://user:PASSWORD@host:5432/n8n"
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                        LE MOT DE PASSE EST EN CLAIR
```

4 problèmes :

1. Le mot de passe apparaît dans l'historique du shell
2. On court-circuite TOUTE la validation applicative de n8n
3. Une écriture directe peut CORROMPRE les données
(workflow_entity, credentials_entity, execution_entity
sont fortement couplées)
4. Aucune vérification de propriété : on peut modifier
le workflow de quelqu'un d'autre

Règles :

```
# ✓ Lecture seulement
n8ncli logs <wf> --db-url "postgresql://..."

# ✓ Connexion chiffrée
n8ncli logs <wf> --db-url "postgresql://...?sslmode=require"

# ✗ JAMAIS d'écriture en production
n8ncli datatables update <table> --data '...' --db-url "..."

# ✓ Utiliser une variable pour éviter l'historique
$env:PG_URL = "postgresql://..."
n8ncli logs <wf> --db-url $env:PG_URL
```

Risque 2 : `publish` — l'activation

```
n8ncli publish "Mon Workflow.workflow.ts"
```

Conséquences immédiates :

- Le cron commence à tourner
- Les webhooks répondent
- Les emails partent
- Les appels API se font
- Chaque exécution est enregistrée

Protection :

```
n8ncli status          # vérifier l'état AVANT
n8ncli diff --semantic  # relire les différences
n8ncli test "Mon Workflow.workflow.ts" # tester AVANT
n8ncli unpublish "Mon Workflow.workflow.ts" # filet de sécurité
```

Risque 3 : `exec --mode production`

```
n8ncli exec <file> --mode production --input '{"email":"real@client.com"}'
```

Un test en production **envoie vraiment** des emails, **appelle vraiment** les API, **écrit vraiment** en base.

Protection :

```
# Utiliser --mode manual et des données de test
n8ncli test <file> --pin-data <fichier>
```

Risque 4 : `--include-data` et les pin data

```
n8ncli execution <file> <exec-id> --include-data
```

Affiche les **payloads bruts** de l'exécution. Si le workflow traite des emails clients, tu les vois. Et si tu colles ça dans une conversation, **fuite**.

Règle : données de test synthétiques uniquement.

```
{
  "json": {
    "email": "test@example.com",
    "userId": "user_12345",
    "cardNumber": "4242424242424242"
  }
}
```

Risque 5 : `live` — la boucle automatique

```
n8ncli live --interval 30
```

Risque 1 : je pousse une modification non testée sans m'en apercevoir

Risque 2 : la boucle tourne en fond et consomme de la bande passante

Risque 3 : je ne sais plus ce qui a été poussé

Risque 4 : conflit en boucle entre deux environnements

Protection :

```
n8ncli live --foreground      # voir ce qui se passe
n8ncli live --status          # état
n8ncli live --stop            # TOUJOURS savoir l'arrêter
```

Risque 6 : `npx -y` — la supply chain

```
"mcpCommand": "npx -y n8n-mcp"
```

`npx -y` **télécharge et exécute** le paquet depuis le registre npm à **chaque appel** s'il n'est pas présent. Si le paquet est compromis un jour, tu l'exécutes sans le savoir.

Protection :

```
# Installer une fois, exécuter localement
npm install -g n8n-mcp
# puis utiliser : "mcpCommand": "n8n-mcp"
```

Module 23 — La checklist de sécurité

Avant de commiter

- ☐ `n8ncli validate --lint --fail-on-warnings`
- ☐ `git status` ← relire chaque fichier
- ☐ `git diff --cached` ← relire les changements
- ☐ `grep` des motifs de secrets
- ☐ pas de données réelles dans les workflows
- ☐ pas de pin data réel
- ☐ `.env` bien gitignoré

Avant de pousser

- ❑ `n8ncli status` ← savoir ce qui change
- ❑ `n8ncli diff --semantic` ← relire les différences
- ❑ `n8ncli push --dry-run` ← simuler
- ❑ Le workflow est-il activé ? ← si oui, plus de prudence
- ❑ Les credentials sont-ils correctement liés ?

Avant de publier

- ❑ `n8ncli test <file>` ← testé avec des données mockées
- ❑ Le cron est-il correct ? ← pas « chaque minute » par erreur
- ❑ Les webhooks sont-ils protégés ? ← authentification
- ❑ Un unpublish est-il possible ? ← filet de sécurité

Le mantra

```
||
|| status → validate → diff → dry-run → push → test → publish ||
||
|| Chaque étape est LOCALE, GRATUITE et SÛRE. ||
||
|| Sauter une étape = prendre un risque pour gagner 30 sec. ||
||
||
```

PARTIE 5 — PRATIQUE

Module 24 — Exercices guidés

Exercice 1 : Premiers pas (5 min)

```
# 1. Où est n8ncli ?  
Get-Command n8ncli | Select-Object Source  
  
# 2. Quelle version ?  
n8ncli --version  
  
# 3. Aide générale  
n8ncli --help  
  
# 4. État actuel  
n8ncli status  
  
# 5. État en JSON  
n8ncli status --json
```

Question : pourquoi `status --json` est-il plus utile dans un script ?

Exercice 2 : Explorer les références (5 min)

```
# Lire l'index des exemples  
Get-Content n8n/references/index.yaml  
  
# Lire le workflow « API Webhook Template »  
Get-Content "n8n/references/workflow_examples_ref/API Endpoints/API Webhook Template.workflow.ts"
```

Question : dans ce fichier, que fait `newCredential('API Key', 'hmsiyDLCgLgKDP17')` ? Est-ce un secret ?

Exercice 3 : Comprendre ton vrai workflow (10 min)

```
Get-Content "n8n/workflows/Assistant de Trajet - Recapitulatif Matinal.workflow.ts"
```

Questions :

1. Quel est le cron ? Que signifie `30 9 * * 1-5` ?

2. Quelle API est appelée pour le trafic ?
3. Que fait le nœud `Analyse Trajet` ?
4. Pourquoi y a-t-il un nœud `IF` avant l'envoi du mail ?
5. Le workflow est-il activé ? Comment le savoir ?

Exercice 4 : Valider et découvrir (10 min)

```
# 1. Valider avec les standards
n8ncli validate --lint

# 2. Voir les changements
n8ncli status
n8ncli diff --semantic

# 3. Simuler un push
n8ncli push --dry-run

# 4. Chercher un nœud
n8ncli nodes search http

# 5. Lire la doc d'un nœud
n8ncli nodes doc n8n-nodes-base.httpRequest
```

Exercice 5 : Créer un workflow (20 min)

Créer le fichier `n8n/workflows/Daily Backup Check.workflow.ts` :

```
import { workflow, node, trigger } from '@n8n/workflow-sdk';

const start = trigger({ type: 'n8n-nodes-base.scheduleTrigger', version: 1.4 });

export default workflow('Daily Backup Check')
  .description('Runs every morning and verifies the backup job succeeded.')
  .add(start);
```

Puis :

```
n8ncli validate --lint
```

Exercice 6 : Sécurité (15 min)

```
# 1. Vérifier ce qui est tracké par Git
git ls-files | Select-String -Pattern "env|credential|secret"

# 2. Chercher des secrets potentiels
git grep -nE "(sk-|api[_-]?key|password|Bearer )" -- "*.workflow.ts"

# 3. Vérifier les fichiers locaux sensibles
Get-Content .gitignore

# 4. Où sont les secrets ?
Get-Content "$env:USERPROFILE\.n8ncli-global.json" | ConvertFrom-Json | Get-Member
```

Questions :

1. Un secret est-il dans le dépôt ?
2. `sync-state.json` et `unconfigured-credentials.json` sont-ils gitignores ? Pourquoi ?

Module 25 — Le mémo

Les 5 commandes à connaître par cœur

<code>n8ncli status</code>	# quoi a changé ?
<code>n8ncli validate --lint</code>	# est-ce correct ?
<code>n8ncli diff --semantic</code>	# quelles différences ?
<code>n8ncli push</code>	# déployer
<code>n8ncli publish</code>	# activer

Les codes de sortie à retenir

```
0 = succès
1 = erreur
2 = validation échouée → NE PAS pousser
3 = conflit → résolution manuelle
```

Le cycle de vie

```
Instance —pull→ Code local —push→ Instance
      |
      | validate
      |
      | test
      |
      | publish (activer)
```

Les 3 règles de sécurité

1. UN SECRET N'EST JAMAIS DANS UN FICHIER VERSIONNÉ
2. `--db-url` EN LECTURE SEULE, JAMAIS EN ÉCRITURE
3. TOUJOURS `--dry-run` AVANT D'ÉCRIRE, TOUJOURS

Les fichiers clés

<code>C:\Users\ema\.n8ncli-global.json</code>	← SECRETS (hors Git)
<code>n8n/config/n8n-cli.json</code>	← config projet (dans Git)
<code>n8n/config/n8n-standards.json</code>	← règles de nommage
<code>n8n/config/sync-state.json</code>	← état (gitignoré)
<code>n8n/workflows/**/*.workflow.ts</code>	← LE CODE

Le vocabulaire

Terme	Signification
Shell	L'interpréteur de commandes (PowerShell)
Terminal	La fenêtre qui affiche
Argument	Un paramètre positionnel
Option / Flag	Un paramètre avec tiret
STDIN / STDOUT / STDERR	Les 3 flux d'un programme
Code de sortie	Le verdict (0 = succès)
Pipe	qui chaîne les commandes
Redirection	> qui stocke la sortie
PATH	Où le shell cherche les programmes
Workflow	Le flux complet dans n8n
Node / Nœud	Une unité de travail
Trigger	Le déclencheur (début)
Execution	Une exécution du workflow
Item	Un objet JSON qui circule
Credential	Un secret stocké côté serveur
Pull / Push	Instance → local / local → instance
Publish	Activer un workflow
MCP	Le protocole de connexion de n8ncli

Les 5 idées à retenir

1. Une CLI = texte + codes de sortie → scriptable, donc automatisable
2. Un workflow n8n = un graphe de nœuds qui traite des « items » JSON
3. n8n CLI transforme les workflows en TypeScript → Git, diff, relecture
4. Le cycle : status → validate → diff → dry-run → push → test → publish
5. Les secrets vivent chez n8n, jamais dans un fichier versionné

Limite connue de cette installation

Ta configuration pointe vers `https://ema1999.app.n8n.cloud` sans clé API REST ni accès base de données.
Conséquence concrète :

`n8ncli push` échoue sur la mise à jour d'un workflow existant et affiche :

```
Failed to update workflow ... REST API, PostgreSQL database, and MCP tools were unable to write changes.
```

Ce qui fonctionne :

Opération	État
Lecture (<code>pull</code> , <code>status</code> , <code>diff</code>)	✓ fonctionne
Recherche MCP	✓ fonctionne
Création d'un workflow	✓ via MCP
Mise à jour d'un workflow existant	X échoue
Publication	X dépend du MCP par workflow

Le contournement consiste à piloter le serveur MCP directement depuis un script Node :

```
Transport : Streamable HTTP
Endpoint   : <instanceUrl>/mcp-server/http
Auth       : Authorization: Bearer <accessToken>
Outils utiles : create_workflow_from_code, publish_workflow,
                update_workflow, move_workflows_to_folder,
                create_folder, list_credentials
```